Gestor de Ventas – Funciones, Procedimientos, Triggers y Eventos (CASO PRÁCTICO). UD06

Bases de datos

Silvio aguilar

Introducción

La empresa **InfoTech Store** registra cada venta en la tabla ventas.
Necesita:

1. **Calcular automáticamente** el importe con descuento de cada venta (los descuentos pueden cambiar).
2. **Auditar** toda inserción en ventas (fecha-hora, usuario, json del registro).
3. **Resumir diariamente** el total facturado y el número de ventas del día anterior.

1. Crear tablas

- Lo primero que haremos será crear la base de datos con el comando CREATE DATABASE infoTech;
- Y ahora las tabas que de hecho nos las da el enunciado:
- a) Crea las tablas ventas, auditoria_ventas y resumen_diario con la estructura indicada:

```
CREATE TABLE ventas (
    id              INT AUTO_INCREMENT PRIMARY KEY,
    fecha_venta     DATETIME NOT NULL,
    producto        VARCHAR(60),
    unidades        INT     NOT NULL,
    precio_unitario DECIMAL(10,2) NOT NULL,
    descuento       TINYINT  DEFAULT 0          -- porcentaje 0-100
);
```

```
CREATE TABLE auditoria_ventas (
        id_audit   BIGINT AUTO_INCREMENT PRIMARY KEY,
        id_venta   INT,
        usuario        VARCHAR(50),
        instante   DATETIME,
        datos_json JSON
);

CREATE TABLE resumen_diario (
        fecha_resumen   DATE PRIMARY KEY,
        total_ventas   INT,
        importe_bruto   DECIMAL(12,2),
        importe_neto DECIMAL(12,2)
);
```

2. Ejercicios

- Bien ahora que tenemos la base de datos y las tablas creadas podemos jugar con ellas y hacer los ejercicios:

- b) Implementa la **función** fn_total_con_descuento(cantidad DECIMAL(10,2), porc TINYINT)
- Devuelve el importe neto (cantidad − porcentaje).
  • Etiquetas: DETERMINISTIC NO SQL.

```
DELIMITER $$

CREATE FUNCTION fn_total_con_descuento(cantidad DECIMAL(10,2), porc TINYINT)
```

```
RETURNS DECIMAL(10,2)
DETERMINISTIC
NO SQL
BEGIN
        DECLARE neto DECIMAL(10,2);

  SET neto = cantidad - (cantidad * porc /100);

  RETURN neto;
END$$
```

- c) Implementa el **procedimiento** sp_insertar_venta(...) con los parámetros de la tabla ventas.
  Dentro de una transacción:
  · Inserta la venta aplicando la función del apartado b.
  · Registra en auditoria_ventas.
  • Controla cualquier error con DECLARE EXIT HANDLER FOR SQLEXCEPTION → ROLLBACK.

```
DELIMITER $$

CREATE PROCEDURE sp_insertar_venta(
  p_fecha_venta DATETIME,
  p_producto VARCHAR(60),
  p_unidades INT,
  p_precio_unitario DECIMAL(10,2),
```

```
    p_descuento TINYINT
)
BEGIN
        DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
                ROLLBACK;
        END;


    START TRANSACTION;
                INSERT INTO ventas(fecha_venta, producto, unidades, precio_unitario, descuento)
            VALUES(p_fecha_venta, p_producto, p_unidades, p_precio_unitario, p_descuento);


      INSERT INTO auditoria_ventas(id_venta, usuario, instante, datos_json)
      VALUES(
                        LAST_INSERT_ID(),
        USER(),
        NOW(),
        JSON_OBJECT(
        'fecha_venta', p_fecha_venta,
        'producto', p_producto,
        'unidades',p_unidades,
        'precio_unitario', p_precio_unitario,
        'descuento', p_descuento
```

```
        )

      );

  COMMIT;
END$$
DELIMITER ;
```

- d) Crea un **trigger** trg_audit_ventas AFTER INSERT sobre ventas que inserte en auditoria_ventas si el procedimiento anterior **no** fue utilizado (por ejemplo, inserciones directas de otros sistemas).

```
DELIMITER $$

CREATE TRIGGER trg_audit_ventas
AFTER INSERT ON ventas
FOR EACH ROW
BEGIN
        INSERT INTO auditoria_ventas(id_venta, usuario, instante, datos_json)
  VALUES(
    NEW.id,
    USER(),
    NOW(),
    JSON_OBJECT(
                    'fecha_venta', NEW.fecha_venta,
      'producto', NEW.producto,
```

```
            'unidades', NEW.unidades,
            'precio_unitario', NEW.precio_unitario,
            'descuento', NEW.descuento
        )
    );

END$$
DELIMITER ;
```

- e) Activa el planificador y crea un **evento** ev_resumen_diario que, cada día a las 03:00, genere (o actualice) el resumen para la fecha del día anterior.

  - Lo primero es activar el planificador de eventos si no lo tenemos activo con el comando: SET GLOBAL event_scheduler = ON;

```
DELIMITER $$
CREATE EVENT ev_resumen_diario
ON SCHEDULE EVERY 1 DAY
STARTS '2026-01-01 03:00:00'
DO
BEGIN
    INSERT INTO resumen_diario(fecha_resumen, total_ventas, importe_bruto, importe_neto)
    SELECT
        DATE(fecha_venta),
        COUNT(*),
        SUM(unidades * precio_unitario),
        SUM(fn_total_con_descuento(unidades * precio_unitario, descuento))
```

```
    FROM ventas
    WHERE DATE(fecha_venta) = CURDATE() - INTERVAL 1 DAY
    GROUP BY DATE(fecha_venta)
    ON DUPLICATE KEY UPDATE
      total_ventas  = VALUES(total_ventas),
      importe_bruto = VALUES(importe_bruto),
      importe_neto  = VALUES(importe_neto);
END$$
DELIMITER ;
```

- f) Ejecuta al menos **tres llamadas** al procedimiento con valores diferentes y, a continuación, muestra los contenidos de las tres tablas.

  - Vamos a hacer tres llamadas al procedimiento sp_insertar_venta con equipo informático, por ejemplo:

    ```
    CALL sp_insertar_venta('2026-03-06 04:50:00', 'Pantalla', 10, 599.99, 30);
    CALL sp_insertar_venta('2026-03-06 04:50:00', 'Torre', 10, 1200.99, 30);
    CALL sp_insertar_venta('2026-03-06 04:50:00', 'Portatil', 30, 2500.99, 30);
    ```

Y ahora vemos el resultado si ha ido todo correcto:

SELECT * FROM ventas;

Result Grid | Filter Rows: | Edit: | Export/Import:

id	fecha_venta	producto	unidades	precio_unitario	descuento
1	2025-01-10 10:00:00	Teclado	2	49.99	10
2	2026-03-06 04:23:37	Ratón	1	25.00	5
3	2026-03-06 04:50:00	Portatil	30	2500.99	30
4	2026-03-06 04:50:00	Portatil	30	2500.99	30
5	2026-03-06 04:50:00	Portatil	30	2500.99	30
6	2026-03-06 04:50:00	Portatil	30	2500.99	30
7	2026-03-06 04:50:00	Portatil	30	2500.99	30
8	2026-03-06 04:50:00	Torre	10	1200.99	30
9	2026-03-06 04:50:00	Pantalla	10	599.99	30
NULL	NULL	NULL	NULL	NULL	NULL

SELECT * FROM auditoria_ventas;

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id_audit	id_venta	usuario	instante	datos_json
1	1	root@localhost	2026-03-06 04:09:18	{"producto": "Teclado", "unidades": 2, "descuen...
2	2	root@localhost	2026-03-06 04:23:37	{"producto": "Ratón", "unidades": 1, "descuent...
3	3	root@localhost	2026-03-06 04:46:11	{"producto": "Portatil", "unidades": 30, "descue...
4	3	root@localhost	2026-03-06 04:46:11	{"producto": "Portatil", "unidades": 30, "descue...
5	4	root@localhost	2026-03-06 04:47:56	{"producto": "Portatil", "unidades": 30, "descue...
6	4	root@localhost	2026-03-06 04:47:56	{"producto": "Portatil", "unidades": 30, "descue...
7	5	root@localhost	2026-03-06 04:47:58	{"producto": "Portatil", "unidades": 30, "descue...
8	5	root@localhost	2026-03-06 04:47:58	{"producto": "Portatil", "unidades": 30, "descue...
9	6	root@localhost	2026-03-06 04:47:59	{"producto": "Portatil", "unidades": 30, "descue...
10	6	root@localhost	2026-03-06 04:47:59	{"producto": "Portatil", "unidades": 30, "descue...
11	7	root@localhost	2026-03-06 04:48:00	{"producto": "Portatil", "unidades": 30, "descue...
12	7	root@localhost	2026-03-06 04:48:00	{"producto": "Portatil", "unidades": 30, "descue...
13	8	root@localhost	2026-03-06 04:48:29	{"producto": "Torre", "unidades": 10, "descuent...
14	8	root@localhost	2026-03-06 04:48:29	{"producto": "Torre", "unidades": 10, "descuent...
15	9	root@localhost	2026-03-06 04:48:35	{"producto": "Pantalla", "unidades": 10, "descu...
16	9	root@localhost	2026-03-06 04:48:35	{"producto": "Pantalla", "unidades": 10, "descu...
NULL	NULL	NULL	NULL	NULL

SELECT * FROM resumen_diario;

Result Grid | Filter Rows: | Edit: | Export/Im

fecha_resumen	total_ventas	importe_bruto	importe_neto
2026-03-06	8	393183.30	275234.56
NULL	NULL	NULL	NULL